Neural networks and regression

Name: Rinkle Sharma
Student ID: 67784

This report covers gradient descent, a manual backpropagation calculation, tensor operations and two PyTorch training examples. The final exercise uses physical measurements from the UCI Abalone dataset to predict age. The accompanying notebook contains the code, numerical tables and plots for each exercise.

1. Running the gradient-descent example

For the first example, the loss is $C(w) = w^2$ and its derivative is $2w$. The update is therefore $w_{new} = w - \eta \cdot 2w$. With an initial weight of 10 and a learning rate of 0.2, each update multiplies the weight by 0.6. After ten updates, the weight is 0.060466176. The result agrees with the direct calculation 10×0.6^{10}.

The lecture calls this SGD, although it uses an exact gradient. Strictly speaking, this is deterministic gradient descent. The regression exercises also include a stochastic version that shuffles the observations and updates the weights after each individual example.

2. Learning rate and number of iterations

The second function is $C(w) = w^4 - 5w^2 - 3w$, with derivative $C'(w) = 4w^3 - 10w - 3$. The experiments cover three starting values (−2, 0 and 2), six learning rates (0.001, 0.01, 0.05, 0.1, 0.2 and 0.5), and three iteration limits (10, 100 and 1000). This gives 54 runs.

The plots show why the learning rate matters. Starting from zero, a rate of 0.001 makes slow progress, while 0.01 and 0.05 approach the minimum sooner. A rate of 0.1 produces persistent oscillations, and 0.5 diverges. The function has two local minima, so the starting weight can also change which solution is reached.

Increasing the iteration limit helps a stable run that has not yet converged. It does not repair an excessive learning rate. For this reason, the results table includes the final gradient magnitude as well as the weight and loss: a small update caused by a tiny learning rate does not necessarily mean that the minimum has been reached.

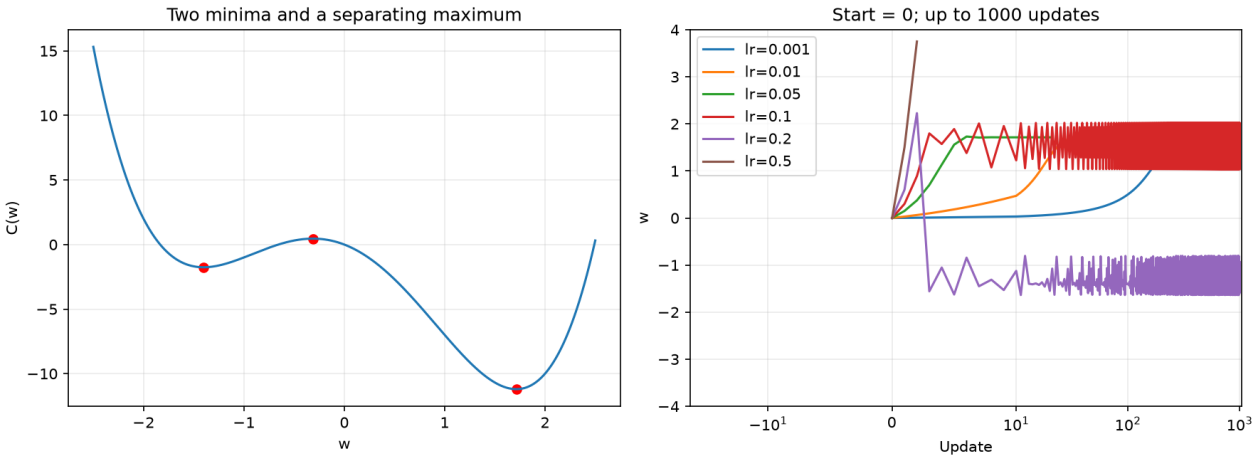


Figure 1. The quartic function and the weight trajectories for different learning rates, starting at zero.

3. Linear regression

The lecture dataset contains six pairs of observations:

x	5	15	25	35	45	55
y	5	20	14	32	22	38

Gradient descent gives the fitted function **y = 5.628223 + 0.540129x**. The notebook plots this line over the observed points and compares its coefficients with the exact least-squares solution. The small difference between the two fits comes from the stopping tolerance. Since all six points are used for fitting, the error here describes the training data.

The supplied temperature example, ftoc2.ipynb, was also run with its original settings: Adam, MAE loss, a learning rate of 0.02 and 100 epochs. Despite the assignment's reference to SGD, the active optimizer in that notebook is Adam; its SGD code is commented out.

A separate version uses SGD, scaled inputs and outputs, MSE loss and 5000 epochs. After converting the coefficients back to their original units, it gives **F = 1.799994C + 32.000241**. This closely matches the known conversion F = 1.8C + 32. The dataset is noiseless, so such a close fit is expected. Because the optimizer, scaling, loss and training duration all differ, this comparison does not measure the effect of the optimizer alone.

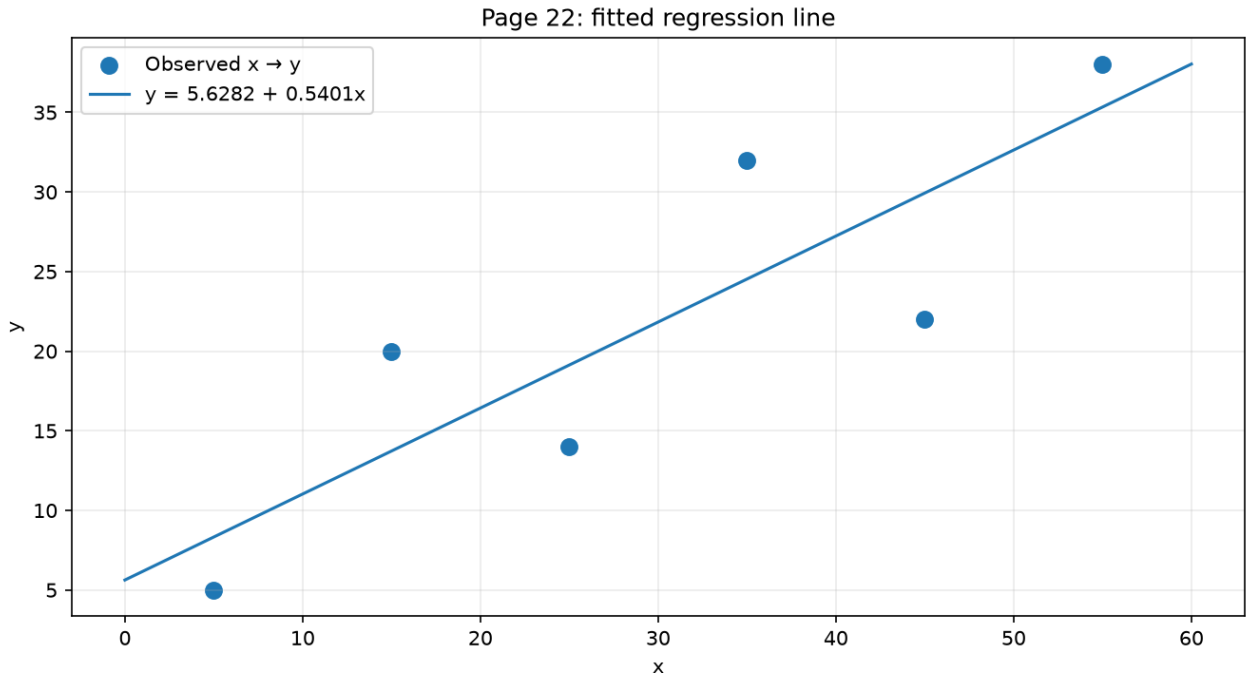


Figure 2. The six lecture observations and fitted regression line.

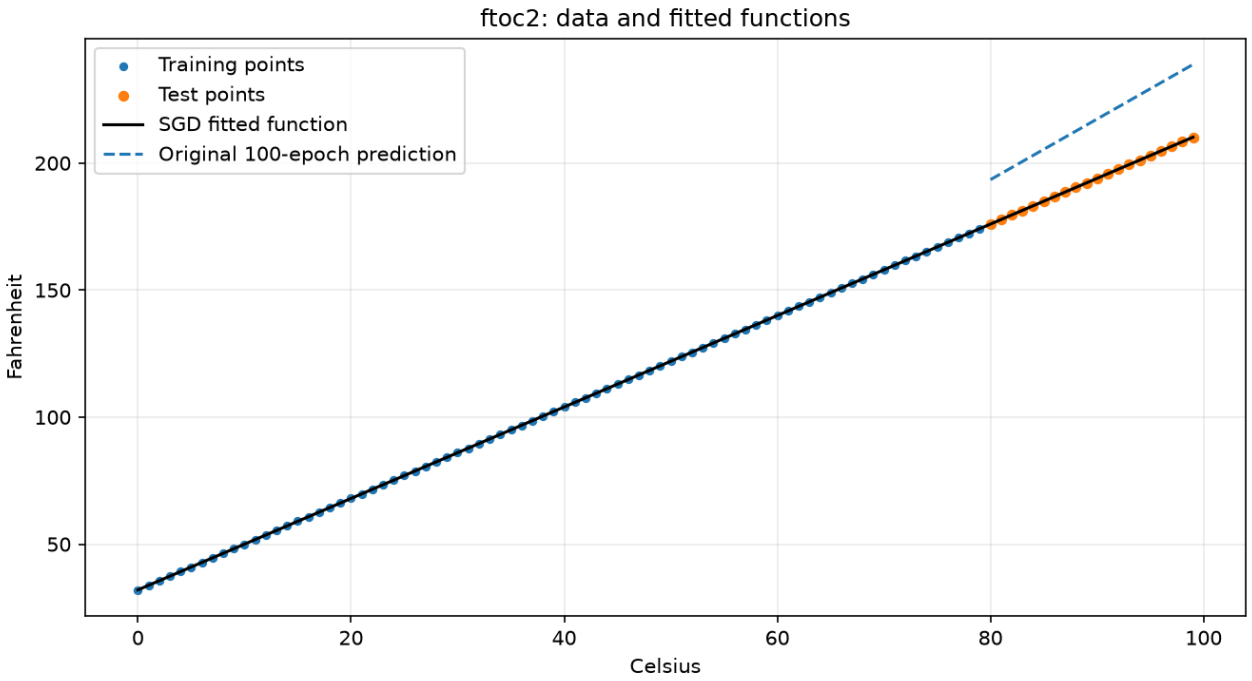


Figure 3. Temperature observations, the scaled SGD fit and the original 100-epoch predictions.

4. Manual forward pass and backpropagation

The manual calculation uses the lecture's two-input, two-hidden-neuron network with a sigmoid output, but starts with different weights. The hidden weight rows, including their biases, are [0.1, 0.2, −0.3] and [−0.2, 0.4, 0.1]. The output weights are [−0.1, 0.3, −0.2], again with the bias first. The input is [1, 0], the target is 1 and the learning rate is 0.2.

The hidden weighted sums are 0.3 and 0.2. Applying the sigmoid function and passing these activations to the output gives a network response of **0.490592599**. Backpropagation calculates the output delta and then the two hidden deltas. Each connection is updated using the delta of its destination neuron and the activation of its source neuron. Bias connections use a constant input of 1.

After updating all nine weights, the response becomes **0.501082446**, moving toward the target and reducing the error. The weights connected to the second input remain unchanged because that input is zero. The notebook shows every intermediate value and checks all nine updates against PyTorch autograd. The hidden deltas are calculated with the old output weights, before any weight updates are applied.

5. Tensor operations

The tensor examples cover addition, subtraction, elementwise multiplication, division and matrix multiplication. They also show how reshape, view, stack, squeeze, unsqueeze and permute change tensor dimensions. The dimensionality table appears on PDF page 46, immediately after the basic operations on page 45.

The examples distinguish changing a tensor's shape from changing its values. Reshaping preserves the number of elements; stacking adds a new dimension; and permuting reorders the axes. A shared-storage example shows that changing a value through a view also changes the original tensor. A transposed tensor demonstrates why view can fail when its strides are incompatible, while reshape can make a copy.

6. PyTorch reproduction of the XOR example

The first PyTorch run uses the exact weights from slide 35, the input [1, 0], target 1, sigmoid activations and SGD with a learning rate of 0.2. One update changes the output from **0.633564545 to 0.644935592** and reduces the half-squared error from **0.067137471 to 0.063035367**. The resulting weights agree with the independent NumPy calculation.

A second run trains a freshly initialized network on all four XOR input patterns. It uses the same 2-2-1 topology, full-batch SGD, a learning rate of 1 and 20,000 epochs. All four training patterns are classified correctly when the output is thresholded at 0.5. This full truth-table run is separate from the single-example update above. The slide-35 calculation can also be run directly using train_xor.py.

7. Predicting abalone age

The real-data exercise uses the UCI Abalone dataset, which contains 4177 observations and eight predictors. The target is age in years, calculated as Rings + 1.5 according to the dataset documentation. Rings is excluded from the inputs because it directly determines the target.

The preparation stage checks data types, missing values and duplicate rows. The distributed data have no missing entries. Nonpositive heights are treated as missing because they are physically questionable. Numeric columns use median imputation and standardization, while the categorical Sex column uses one-hot encoding. All imputation and scaling statistics are learned from the training set. The target is also standardized for training and converted back to years before evaluation.

The data are split into 60% training, 20% validation and 20% test observations, using a fixed random seed. Six neural-network configurations are compared on validation MAE:

Configuration	Hidden layers	Learning rate	Best epoch	Validation MAE (years)
Adam, small network	32	0.001	104	1.4779
Adam, two hidden layers	64, 32	0.001	31	1.4627
Adam, higher learning rate	64, 32	0.01	8	1.4579
Adam with weight decay	64, 32	0.003	29	1.4491
SGD with momentum	64, 32	0.03	57	1.4338
Adam, 30-epoch limit	64, 32	0.001	22	1.4797

The selected network has ten prepared inputs, hidden layers of 64 and 32 ReLU units, and one linear output. It uses SGD with momentum 0.9, weight decay 0.001 and minibatches of 128. Early stopping ends the run at epoch 107 and restores the weights from epoch 57, which had the lowest validation MAE. The test set is evaluated only after this selection.

Model	Test MAE (years)	Test MSE (years²)	Test MAPE (%)
Training-mean baseline	2.3851	10.8357	22.71
Ridge regression	1.6188	4.9551	13.98
Selected neural network	1.4653	4.5262	12.23

The neural network performs better than both baselines on these metrics. Its test RMSE is **2.1275 years** and its R^2 is **0.5819**, based on 836 test observations. MAE describes the average absolute error in years. MSE gives more weight to large errors, while MAPE expresses the error relative to the animal's actual age. All ages are positive, so MAPE has no division-by-zero problem here.

The training curves show that the final epoch is not necessarily the best checkpoint. A higher learning rate reaches useful results sooner but can fluctuate. The short Adam run also performs slightly worse than the longer run with the same architecture and learning rate. The experiment table compares complete configurations; where several settings differ, their individual effects cannot be separated from this comparison alone.

The residual plots show a remaining weakness: older animals tend to be underestimated. The model is useful as an estimate, but its predictions should not be treated as exact ages. These results come from one historical dataset and one split. Repeated cross-validation or data from another location would be needed to judge how consistently the model performs.

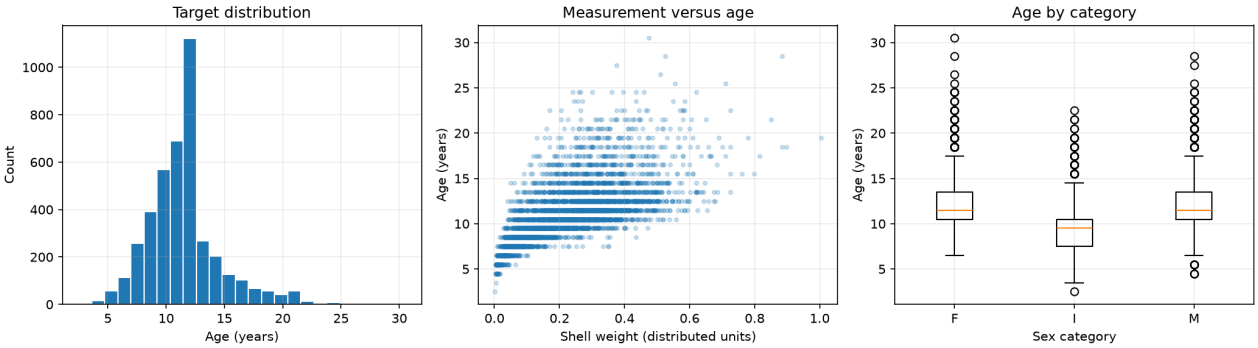


Figure 4. Age distribution and relationships between age and selected predictors.

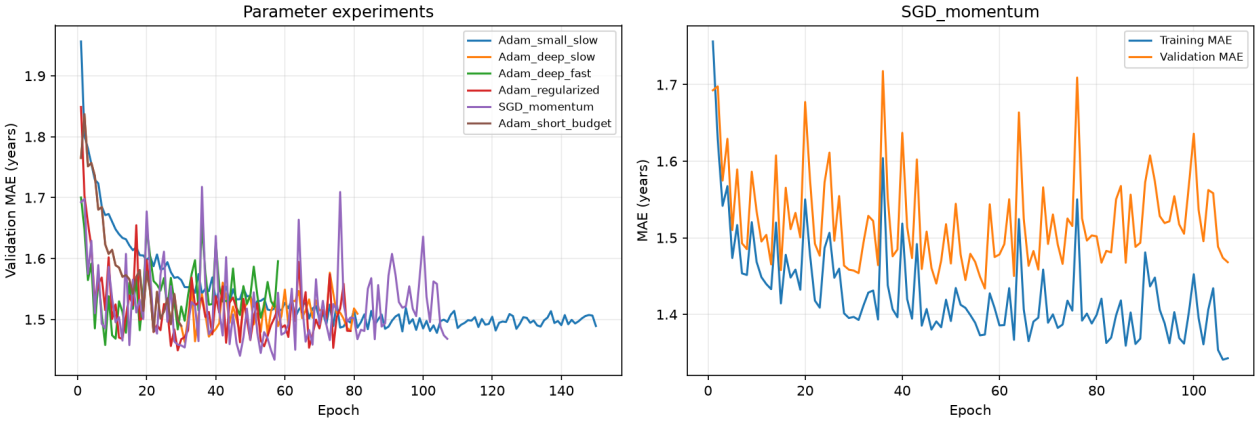


Figure 5. Validation MAE across six configurations and the training history of the selected network.

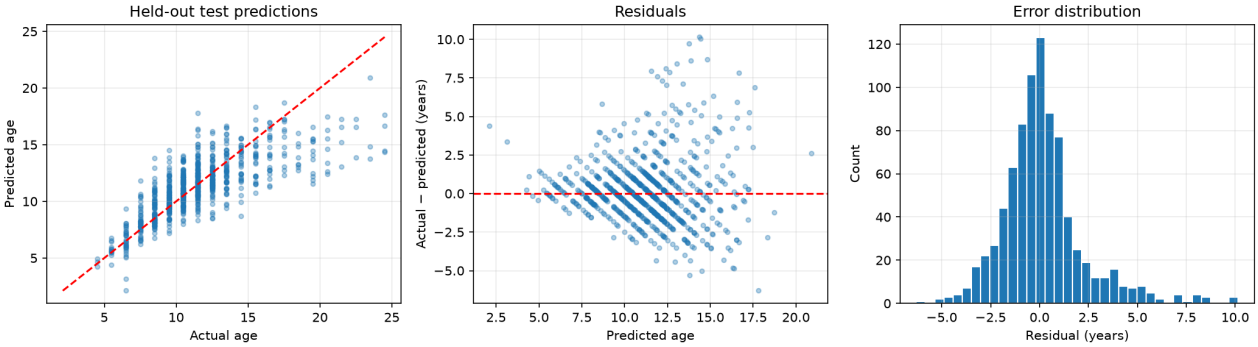


Figure 6. Predictions and residuals for the held-out Abalone test set.

Sources

• NN_English.pdf, lecture pages 19-24, 34-36 and 43-46.

• ftoc2.ipynb, the Celsius-to-Fahrenheit regression example.

• UCI Abalone dataset, DOI: 10.24432/C55C7W, retrieved 24 October 2026. Creators: Warwick Nash, Tracy Sellers, Simon Talbot, Andrew Cawthorn and Wes Ford. The dataset is distributed under CC BY 4.0; the original data and attribution are included in the accompanying package.